

Free Form Testing Agent --- Master Project Plan

Document Type: Project Roadmap and Scope\

Project: Free Form Testing Agent (FFTA)\

Primary Language: Python\

Initial Target Platform: Windows Desktop Applications\

Initial Reference Application: Windows Calculator\

Initial AI Runtime Direction: Local/offline models through Ollama\

Current Phase: Phase 2 --- Autonomous Exploration and Replay\

Last Updated: July 2026

1. Purpose of This Document

This document is the master planning reference for the Free Form Testing Agent.

Its purpose is to answer four questions whenever development resumes:

1. What are we building?
2. Why are we building it?
3. What have we completed?
4. What should we build next?

The project is intentionally being developed phase by phase. Each phase adds one major capability while preserving and testing everything built earlier.

The long-term goal is not to create another scripted UI automation framework. The goal is to build an autonomous testing agent that can observe an unknown application, explore it, learn its behavior, build a reusable model of the application, discover failures, reproduce them, and eventually reason about what to test.

2. Project Vision

Traditional automation usually follows this model:

```
Human writes test
↓
Framework executes steps
↓
Assertions pass or fail
```

The Free Form Testing Agent aims for:

```
Application
↓
Agent observes UI
↓
Agent captures state
↓
Agent discovers possible actions
↓
Agent chooses and executes actions
↓
Agent observes the result
↓
Agent learns application behavior
↓
Agent builds an executable state graph
↓
Agent discovers workflows and failures
↓
Agent reproduces bugs
↓
Agent generates reusable tests
```

The final system should be able to receive an application with little or no predefined test script and progressively build knowledge about how that application behaves.

3. Core Mental Model

The entire project is based on one central rule:

STATE + ACTION = NEXT STATE

Example:

```
State S0
Display = 0

+
Click 7
↓
```

```
State S1  
Display = 7
```

A complete application can therefore be represented as a graph:

```
S0 --Click(7)--> S1  
S0 --Click(8)--> S2  
S1 --Click(+)--> S3  
S3 --Click(8)--> S4
```

In this model:

- **States** are nodes.
 - **Actions/Transitions** are edges.
 - **Exploration** discovers the graph.
 - **Replay** executes paths through the graph.
 - **Coverage** measures how much of the graph has been explored.
 - **Bug detection** identifies unexpected or failed states.
 - **AI planning** decides which unexplored paths are most valuable.
-

4. Scope of the Project

4.1 Initial Scope

The first practical version focuses on:

- Windows desktop applications.
- UI Automation based state capture.
- Click-based action discovery and execution.
- Deterministic state hashing.
- State graph construction.
- Autonomous exploration.
- Replay and state recovery.
- Coverage measurement.
- Failure detection.
- Test generation.
- Local AI-assisted planning.

The Windows Calculator is being used as the initial reference application because it provides:

- Many controls.

- Predictable state changes.
- Branching action possibilities.
- Invalid operations.
- Easy visual verification.
- Fast application restart.

The Calculator is only a development target. The architecture must remain generic enough to support other applications later.

4.2 Long-Term Scope

The long-term platform may support:

- Desktop applications.
- Web applications.
- Mobile applications.
- Multi-window workflows.
- Keyboard input.
- Text fields.
- Menus and dialogs.
- Drag and drop.
- Scrolling.
- Visual state understanding.
- Semantic UI understanding.
- Local LLM planning.
- Bug classification.
- Autonomous test generation.
- Persistent knowledge across test sessions.
- Multi-agent exploration.
- Distributed exploration.

These are roadmap goals, not requirements for the current phase.

5. Development Principles

The project follows these rules:

5.1 Build One Capability at a Time

```
Design
↓
Implement
↓
Create Test
↓
Run Against Real Application
↓
Fix Problems
↓
Add to Regression Suite
↓
Move Forward
```

5.2 Every Major Component Must Be Testable

Tests are divided into:

```
Unit Tests
  Pure Python logic

Integration Tests
  Real application interaction

Regression Tests
  Previously discovered bugs and failures
```

The complete system should be checkable with one master test command.

5.3 Keep Components Replaceable

Examples:

```
BFS Explorer
  can later become
DFS Explorer
Coverage Explorer
LLM Explorer

Default Action Filter
  can later become
Application-Specific Filter
Risk-Aware Filter
LLM Filter
```

5.4 Prefer Observable Behavior Over Assumptions

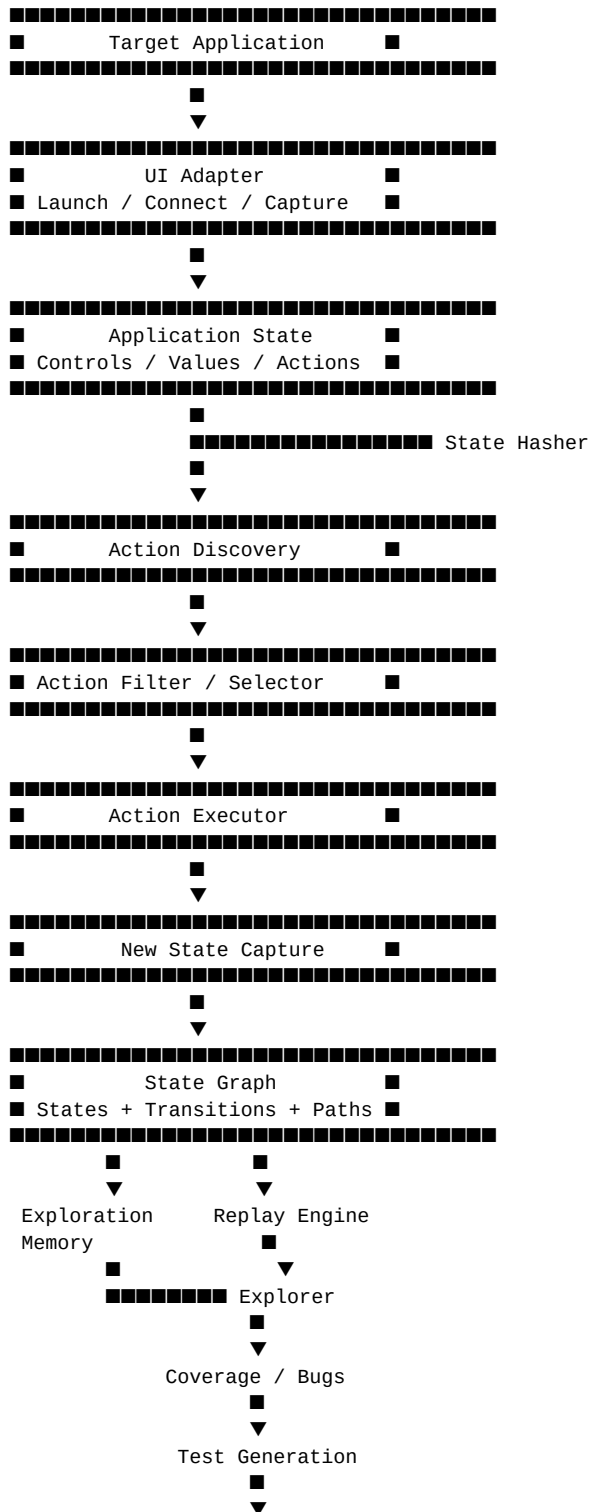
The agent should learn from:

- Captured states.
- Executed actions.
- Actual transitions.
- Replay results.

- Real failures.

It should not assume application behavior unless that behavior has been observed or inferred by a future reasoning layer.

6. High-Level Architecture



7. Phase 0 --- Project Definition and Architecture

Goal

Define what the Free Form Testing Agent is, what problem it solves, and how the system should be divided into components.

Steps

Step 0.1 --- Define the Problem

Understand the limitations of traditional scripted automation:

- Manual test creation.
- High maintenance.
- Limited unknown-path discovery.
- Poor adaptability.
- Weak autonomous behavior.

Step 0.2 --- Define the Core State Model

Establish:

State + Action = Next State

Step 0.3 --- Define the Layered Architecture

Separate:

- Adapters.
- Core models.
- State processing.
- Graph.
- Memory.
- Executor.
- Explorer.
- Replay.
- Coverage.

- Intelligence.

Step 0.4 --- Choose Initial Technology

Initial direction:

- Python.
- Windows UI Automation.
- Pydantic models.
- Local/offline AI through Ollama.
- Initial model direction: qwen2.5-coder : 7b.

Achievement

A scalable architecture was established before autonomous exploration was implemented.

Status

Completed

8. Phase 1 --- Observation, State, Action, Graph, and Testing Foundation

Goal

Give the agent the ability to observe a real application, represent what it sees, execute actions, and remember discovered behavior.

Step 1.1 --- Windows UI Adapter

Objective

Create a boundary between the core agent and the operating system UI automation technology.

Purpose

The adapter is responsible for:

- Launching an application.
- Connecting to its window.
- Finding controls.
- Capturing application state.
- Interacting with the Windows UI layer.

Achievement

The agent can connect to and inspect the Windows Calculator.

Status

Completed

Step 1.2 --- UI Control Representation

Objective

Convert raw Windows UI Automation controls into stable internal models.

Purpose

The rest of the agent should work with project-owned models rather than raw automation objects.

A control may include information such as:

- Automation ID.
- Name.
- Control type.
- Bounding rectangle.
- Enabled/visible state.
- Other useful metadata.

Achievement

The application UI can be represented as structured data.

Status

Completed

Step 1.3 --- Application State Model

Objective

Represent a complete application snapshot.

Current State Model

```
ApplicationState
  state_id
  timestamp
  window_title
  controls
  values
  available_actions
  screenshot_path
  metadata
  state_hash
```

Purpose

A state is the fundamental unit of application knowledge.

Achievement

The agent can capture the current application as a structured state.

Status

Completed

Step 1.4 --- Action Model and Action Discovery

Objective

Convert interactable controls into executable actions.

Purpose

The agent must know what it can do from each state.

Example:

```
Control:
  AutomationId = num7Button
```

Becomes:

```
Action:
  CLICK(num7Button)
```

Achievement

Available actions are stored with each captured state.

Status

Completed

Step 1.5 --- Action Executor

Objective

Execute an Action against the real application.

Purpose

Bridge the internal action model with real UI interaction.

Responsibilities

- Find the target control.
- Perform the requested interaction.
- Return success or failure.
- Support future action types.

Achievement

The agent can execute real clicks on Calculator controls.

Status

Completed

Step 1.6 --- State Hashing

Objective

Determine whether two captured states represent the same logical application state.

Current Hash Inputs

Window Title
+
Serialized Controls
+
Dynamic Values

Purpose

Prevent duplicate state storage and support replay verification.

Achievement

States can be deduplicated using deterministic SHA-256 hashes.

Status

Completed

Step 1.7 --- State Node

Objective

Wrap an `ApplicationState` with graph-related runtime information.

Purpose

Separate the raw state snapshot from graph behavior such as visit tracking.

Achievement

Graph nodes can track repeated visits without modifying the original state identity.

Status

Completed

Step 1.8 --- Transition Model

Objective

Represent an observed state change.

Current Transition

```
Transition
  source_state
  target_state
  action
  success
  duration
```

Purpose

A transition stores exactly how one state led to another.

Achievement

The graph contains executable actions, not only labels.

Status

Completed

Step 1.9 --- State Graph

Objective

Store discovered application behavior.

Purpose

Represent the application as:

```
State Nodes
+
Transition Edges
```

Current Capabilities

- Add unique states.
- Detect duplicate states.
- Add transitions.
- Get neighboring states.
- Print graph.
- Find state paths.
- Find transition paths.

Achievement

Observed application behavior can be stored as an executable knowledge graph.

Status

Completed

Step 1.10 --- Graph Search

Objective

Find routes between known states.

Purpose

Support:

- Navigation.
- Replay.
- Backtracking.
- Workflow reconstruction.
- Future planning.

Current Capabilities

- BFS state path search.
- BFS transition path search.

Achievement

The system can retrieve the sequence of transitions needed to reach a discovered state.

Status

Completed

Step 1.11 --- Exploration Memory

Objective

Remember which actions have already been executed from each state.

Purpose

Avoid repeatedly trying the same action from the same logical state.

Core Model

```
State Hash
  ↓
Executed Action Targets
```

Achievement

The explorer can ask for unexplored actions.

Status

Completed

Step 1.12 --- Testing Infrastructure

Objective

Make the project safe to extend.

Structure

```
tests/
  unit/
  integration/
  regression/
  fixtures/
  run_all_tests.py
```

Purpose

Detect when new development breaks previously working components.

Additional Work

- Shared Calculator fixture.
- Application startup and cleanup.
- Reuse of the current Python interpreter in subprocess tests.
- One-command test execution.

Achievement

The project has a maintainable regression foundation.

Status

Completed

9. Phase 2 --- Autonomous Exploration and State Recovery

Goal

Transform the framework from manually triggered components into an autonomous explorer.

This is the current phase.

Step 2.1 --- Explorer Base Interface

Objective

Define a common contract for exploration strategies.

Purpose

Allow future implementations such as:

- BFS.
- DFS.
- Random.
- Coverage-driven.
- LLM-guided.

Achievement

A common Explorer abstraction exists.

Status

Completed

Step 2.2 --- Exploration Context

Objective

Store runtime information for one exploration session.

Current Responsibilities

- State queue.
- Visited states.
- State count.
- Action count.
- Transition count.

Purpose

Keep temporary exploration state separate from permanent application knowledge.

Achievement

The BFS runtime has a dedicated context object.

Status

Completed

Step 2.3 --- Exploration Result

Objective

Return structured statistics from an exploration run.

Current Information

- States.
- Transitions.
- Actions.
- Duration.

Purpose

Measure and compare exploration runs.

Achievement

Exploration produces a structured result.

Status

Completed

Step 2.4 --- BFS Explorer V1

Objective

Automatically explore the application using a queue-based strategy.

Initial Flow

```

Capture Initial State
  ↓
Add State To Queue
  ↓
Get Unexplored Actions
  ↓
Execute Action
  ↓
Capture New State
  ↓
Store Transition
  ↓
Queue New State

```

Achievement

The agent automatically executed actions and built a graph.

Problem Discovered

The first version treated all discovered actions as safe.

It attempted actions such as:

- Minimize.
- Close.
- Maximize.
- Invalid or unusable controls.

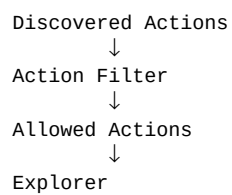
Status

Step 2.5 --- Action Filtering

Objective

Prevent dangerous, invalid, or low-value actions from disrupting autonomous exploration.

Architecture



Initial Blocked Actions

Examples:

- Empty target.
- Close.
- Minimize.
- Maximize.
- Restore.
- Certain memory controls.
- Other known disruptive controls.

Achievement

The explorer stopped minimizing or closing the application during exploration.

Status

Completed V1

Step 2.6 --- Identify the Branch Exploration Problem

Problem

The first BFS implementation executed many actions sequentially on the current live application.

Actual behavior:

```
S0 --A--> S1 --B--> S2 --C--> S3
```

But correct BFS branching requires:

```
      --A--> S1
     /
S0  ----
     \
      --B--> S2
```

To test action B from S0, the real application must first return to S0.

Conclusion

A correct autonomous explorer requires state restoration.

Status

Problem Identified

Step 2.7 --- Replay Engine V1

Objective

Restore a previously discovered state by replaying the known transition path.

Flow

```
Target State Requested
      ↓
Find Transition Path
      ↓
Restart Application
      ↓
Connect To Fresh Window
      ↓
Replay Actions In Sequence
      ↓
Return Recovered Window
```

Achievement

One-step replay was successfully tested:

S0 --Click(7)--> S1

The system:

1. Discovered S1.
2. Restarted Calculator.
3. Replayed Click(7).
4. Returned to the target state.

Status

Completed V1

Step 2.8 --- Replay Verification

Objective

Do not assume that replay succeeded just because every click returned success.

Flow

```
Replay Actions
  ↓
Capture Actual State
  ↓
Generate Actual State Hash
  ↓
Get Expected Target State
  ↓
Compare Hashes
```

Success Condition

actual_state_hash == expected_state_hash

Purpose

Make replay trustworthy enough for autonomous exploration.

Status

Implemented / Current Validation Area

Step 2.9 --- Multi-Step Replay Chain

Objective

Prove that the replay engine can restore a state through multiple transitions.

Planned Test

S0 --7--> S1 --+--> S2 --8--> S3

Then:

```
Restart Application
↓
Replay S0 → S3
↓
Execute 7
↓
Execute +
↓
Execute 8
↓
Verify Final Hash == S3 Hash
```

Purpose

Validate:

- Transition path search.
- Ordered action replay.
- Multi-action execution.
- Final state verification.

Status

Next Immediate Step

Step 2.10 --- Integrate Replay Into BFS Explorer

Objective

Make BFS exploration structurally correct.

Intended Algorithm

```
Queue State
↓
Select State To Explore
↓
Replay Root → Selected State
↓
Verify State
```

```

      ↓
Choose One Unexplored Action
      ↓
Execute Action
      ↓
Capture Result
      ↓
Store State + Transition
      ↓
Queue New State
      ↓
Repeat

```

Purpose

Explore branches from the correct source state instead of continuing from a mutated application.

Expected Result

```

      --7--> S1
      /
S0 ---- --8--> S2
      \
      --9--> S3

```

Status

Planned After Multi-Step Replay

10. Phase 3 --- Reliable Exploration and Coverage

Goal

Make autonomous exploration measurable, controlled, and robust.

Step 3.1 --- Exploration Limits

Objective

Prevent uncontrolled graph growth.

Planned Limits

- Maximum states.
- Maximum actions.
- Maximum depth.

- Maximum duration.
- Maximum failures.
- Per-state action limits.

Achievement Target

Exploration can run safely without becoming infinite.

Step 3.2 --- Coverage Engine

Objective

Measure what the agent has explored.

Planned Metrics

State Coverage

$$\frac{\text{Visited Known States}}{\text{Known States}}$$

Action Coverage

$$\frac{\text{Executed Actions}}{\text{Discovered Actions}}$$

Transition Coverage

$$\frac{\text{Executed Transitions}}{\text{Known Transitions}}$$

Additional Metrics

- New states per minute.
- Duplicate-state rate.
- Failed-action rate.
- Maximum depth.
- Branch coverage.
- Replay success rate.

Achievement Target

The agent can explain how thoroughly it explored the application.

Step 3.3 --- Better Action Selection

Objective

Stop treating all allowed actions as equally valuable.

Planned Strategies

- Priority-based selection.
- New-control preference.
- New-state probability.
- Operator/category diversity.
- Risk scoring.
- Coverage gain scoring.

Achievement Target

The explorer spends more time on actions likely to discover new behavior.

Step 3.4 --- DFS Explorer

Objective

Explore deep workflows.

Purpose

BFS is strong for broad shallow coverage. DFS is useful for:

- Long workflows.
- Nested dialogs.
- Multi-step forms.
- Hidden application paths.

Achievement Target

Support multiple interchangeable exploration strategies.

Step 3.5 --- Cycle and Dead-End Handling

Objective

Recognize repetitive or unproductive behavior.

Planned Detection

- Self-loops.
- Repeated cycles.
- Dead-end states.
- States with no safe actions.
- States with only duplicate outcomes.

Achievement Target

Reduce wasted exploration.

Step 3.6 --- Replay Reliability Improvements

Planned Features

- Retry failed replay actions.
- Wait for UI stabilization.
- Per-action timeout.
- Intermediate state verification.
- Better error reporting.
- Replay diagnostics.
- Partial replay.
- Checkpoints.

Achievement Target

Replay becomes reliable enough for large applications.

11. Phase 4 --- Failure Detection and Bug Intelligence

Goal

Move from exploring behavior to identifying suspicious behavior.

Step 4.1 --- Failure Observation

Objective

Capture technical failures.

Planned Signals

- Action execution failure.
 - Application crash.
 - Window disappearance.
 - Frozen UI.
 - Timeout.
 - Unexpected dialog.
 - Unhandled error message.
-

Step 4.2 --- Unexpected State Detection

Objective

Identify states that appear abnormal even when the application does not crash.

Examples

- Error state.
 - Empty screen.
 - Broken layout.
 - Disabled workflow.
 - Unexpected navigation.
 - Invalid calculation result.
-

Step 4.3 --- Bug Record Model

Planned Information

Bug ID
Source State
Action
Observed State
Expected/Reason
Replay Path
Screenshot
Logs
Severity
Reproducibility
Timestamp

Achievement Target

Every discovered issue becomes structured evidence.

Step 4.4 --- Automatic Reproduction

Objective

Use the graph and replay engine to reproduce a suspected bug.

Flow

```
graph TD
    A[Bug Found] --> B[Store Path]
    B --> C[Restart Application]
    C --> D[Replay Path]
    D --> E[Observe Failure Again]
```

Achievement Target

Separate reproducible bugs from transient failures.

Step 4.5 --- Failure Minimization

Objective

Find the shortest action sequence that still reproduces a bug.

Example

Original:

20 actions → crash

Minimized:

4 actions → same crash

Achievement Target

Produce useful developer-ready reproduction steps.

12. Phase 5 --- Test Case Generation and Regression

Goal

Convert discovered application knowledge into reusable automated tests.

Step 5.1 --- Workflow Extraction

Objective

Convert graph paths into meaningful workflows.

Example:

```
Root
→ Enter Value
→ Choose Operation
→ Enter Value
→ Calculate
```

Step 5.2 --- Test Case Generation

Objective

Generate executable tests from discovered paths.

Generated Test Content

- Setup.
- Action sequence.
- Expected target state.
- Verification.
- Cleanup.

Step 5.3 --- Regression Test Generation

Objective

Turn every confirmed bug into a permanent test.

Flow

```
graph TD; A[Bug Discovered] --> B[Bug Reproduced]; B --> C[Minimal Path Found]; C --> D[Regression Test Generated]; D --> E[Future Builds Rechecked];
```

Step 5.4 --- Test Suite Maintenance

Planned Capabilities

- Detect obsolete tests.
- Update changed state hashes.
- Flag broken replay paths.
- Group tests by workflow.
- Prioritize high-risk tests.

Achievement Target

The agent not only discovers tests but helps maintain them.

13. Phase 6 --- Persistent Knowledge and Session Memory

Goal

Allow the agent to continue learning across multiple runs.

Step 6.1 --- Graph Persistence

Objective

Save states and transitions outside process memory.

Possible Storage

- SQLite.
 - Serialized graph files.
 - Dedicated graph database later.
-

Step 6.2 --- Exploration Session Storage

Store

- Session ID.
 - Application version.
 - Start/end time.
 - States discovered.
 - Actions executed.
 - Bugs found.
 - Coverage changes.
-

Step 6.3 --- Resume Exploration

Objective

Continue from previous knowledge instead of starting from zero.

Flow

```
Load Existing Graph
↓
Identify Unexplored Areas
↓
Replay Required State
↓
Continue Exploration
```

Step 6.4 --- Application Version Comparison

Objective

Compare behavior between builds.

Detect

- New states.
- Removed states.
- Changed transitions.
- Broken workflows.
- New failures.

Achievement Target

Support intelligent regression analysis.

14. Phase 7 --- Intelligent Planning with Local AI

Goal

Add reasoning after the deterministic foundation is reliable.

The AI should guide the system, not replace the tested low-level infrastructure.

Step 7.1 --- Local Model Integration

Initial Direction

Use Ollama with an offline model.

Initial candidate:

qwen2.5-coder : 7b

The exact model can be changed based on:

- Hardware.
- Speed.
- Reasoning quality.

- Tool-use quality.
 - Context size.
-

Step 7.2 --- State Summarization

Objective

Convert large raw UI states into concise semantic descriptions.

Example:

```
Raw:
150 UI controls

Summary:
Calculator in Standard mode.
Display shows 7.
Addition operator selected.
```

Step 7.3 --- AI Action Selection

Objective

Allow the model to choose high-value actions.

Inputs may include:

- Current state summary.
- Unexplored actions.
- Coverage gaps.
- Previous failures.
- Goal.

Output:

```
Choose num0Button because division-by-zero behavior
has not yet been explored.
```

Step 7.4 --- Goal-Directed Exploration

Examples

```
Find calculation errors.
```

Explore all settings.

Try invalid inputs.

Find workflows that produce error dialogs.

Achievement Target

Move from blind crawling to purposeful testing.

Step 7.5 --- Semantic Bug Analysis

Objective

Use AI to reason about whether an observed state is suspicious.

Examples:

- Result appears inconsistent.
 - Error message is unclear.
 - Workflow is blocked.
 - UI state contradicts previous behavior.
-

Step 7.6 --- Natural-Language Test Reports

Objective

Generate useful reports from graph evidence.

Example:

The issue is reproducible in 4 steps.

1. Launch Calculator.
2. Click 7.
3. Click Divide.
4. Click 0.

Observed:

...

Replay success rate:

5/5

15. Phase 8 --- Advanced Autonomous Testing

Goal

Expand beyond basic exploration and single-agent reasoning.

Step 8.1 --- Heuristic Explorer

Use domain-independent heuristics to prioritize actions.

Step 8.2 --- Coverage-Driven Explorer

Choose actions based on expected coverage gain.

Step 8.3 --- Risk-Driven Explorer

Prioritize:

- Destructive workflows.
 - Error-prone areas.
 - Recently changed features.
 - Previously unstable states.
-

Step 8.4 --- Learning From Historical Runs

Use previous sessions to estimate:

- Which actions discover new states.
 - Which paths are unstable.
 - Which areas frequently contain bugs.
-

Step 8.5 --- Multi-Agent Exploration

Possible future roles:

Explorer Agent
Discovers states

Replay Agent
Validates reproducibility

Bug Agent
Analyzes failures

Coverage Agent
Finds unexplored areas

Planner Agent
Chooses goals

Step 8.6 --- Distributed Exploration

Run multiple exploration workers against:

- Different application areas.
 - Different strategies.
 - Different builds.
 - Different environments.
-

16. Phase 9 --- Platform Expansion

Goal

Reuse the core architecture across different UI platforms.

The core model remains:

State + Action = Next State

Only adapters and platform-specific action execution should change.

Step 9.1 --- Web Adapter

Potential support:

- Browser DOM.
- Web controls.
- URLs.

- Network events.
 - Browser screenshots.
-

Step 9.2 --- Mobile Adapter

Potential support:

- Android.
 - iOS.
 - Touch actions.
 - Device state.
 - App lifecycle.
-

Step 9.3 --- Multi-Window Desktop Applications

Support:

- Dialogs.
 - Child windows.
 - Popups.
 - Multiple processes.
 - System file pickers.
-

17. Current Project Status

Completed Foundation

- ✓ Project architecture
- ✓ Windows UI Adapter
- ✓ UI control extraction
- ✓ ApplicationState
- ✓ Action discovery
- ✓ Action execution
- ✓ State hashing
- ✓ StateNode
- ✓ Transition model
- ✓ StateGraph
- ✓ Graph search
- ✓ ExplorationMemory
- ✓ Unit test structure
- ✓ Integration test structure
- ✓ Test runner
- ✓ Application fixtures and cleanup

- ✓ Explorer base
- ✓ ExplorationContext
- ✓ ExplorationResult
- ✓ BFS Explorer V1
- ✓ Default Action Filter
- ✓ Replay Engine V1
- ✓ One-step replay test

Current Validation Work

- Replay final-state verification

Immediate Next Step

- Multi-step replay chain

Next Major Integration

- Replay-aware BFS Explorer

18. Immediate Development Roadmap

The exact near-term order should be:

1. Finish Replay Verification

Confirm:

Expected Hash == Actual Hash

2. Build Multi-Step Replay Test

Validate:

S0 --7--> S1 ---+--> S2 --8--> S3

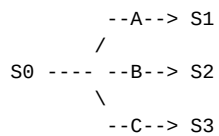
3. Integrate Replay Into BFS

For every queued state:

```
Restart
→ Replay To State
→ Verify
→ Execute One Unexplored Action
→ Capture
→ Store
```

4. Test True BFS Branching

Expected graph:



5. Add Exploration Limits

Prevent uncontrolled execution.

6. Build Coverage Engine

Measure states, actions, transitions, and unexplored areas.

7. Improve Action Selection

Prioritize useful actions.

8. Add DFS Explorer

Validate strategy interchangeability.

9. Add Failure Detection

Capture crashes, errors, and abnormal states.

10. Add Bug Replay and Regression Generation

Turn discovered failures into permanent tests.

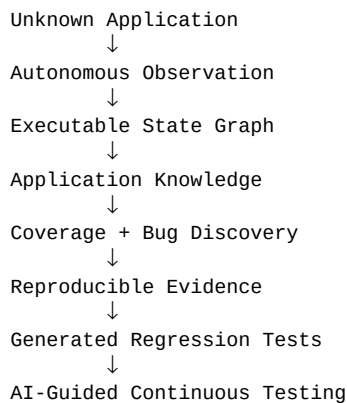
19. What We Are Ultimately Trying to Achieve

The final Free Form Testing Agent should be able to:

1. Receive an application.
2. Launch and observe it.
3. Understand the current UI state.
4. Discover possible actions.
5. Safely execute actions.
6. Observe state changes.

7. Build a graph of application behavior.
8. Avoid repeating known exploration.
9. Restore previously discovered states.
10. Explore branches correctly.
11. Measure coverage.
12. Detect failures and suspicious states.
13. Reproduce bugs automatically.
14. Minimize reproduction paths.
15. Generate regression tests.
16. Persist knowledge between runs.
17. Use local AI to choose intelligent testing goals.
18. Produce human-readable reports.

The desired transformation is:



20. Project Success Criteria

The project can be considered successful when the agent can:

- Explore a previously unseen application with minimal configuration.
- Build a useful graph of states and transitions.
- Restore discovered states reliably.
- Avoid unsafe and repetitive actions.
- Identify unexplored application areas.
- Detect and reproduce failures.
- Generate reusable tests.
- Continue learning across sessions.
- Use local AI for higher-level planning without depending on it for

basic reliability.

21. Development Reminder

When returning to this project after a break, use this order:

1. Read Section 17 – Current Project Status.
2. Read Section 18 – Immediate Development Roadmap.
3. Run the complete test suite.
4. Fix any regression before adding features.
5. Continue only with the next unfinished roadmap step.
6. Update this document when a phase or major step changes.

The current development pointer is:

CURRENT:
Replay verification

NEXT:
Multi-step replay chain

AFTER THAT:
Replay-aware BFS exploration

22. Final Summary

The Free Form Testing Agent is being built as a layered autonomous testing system.

The project started with the ability to observe a Windows application and has progressed to:

```
Observe
↓
Model State
↓
Discover Actions
↓
Execute
↓
Observe Transition
↓
Build Graph
↓
Explore Automatically
↓
Replay Known Paths
```

The next major milestone is:

Correct Branch-Based Autonomous Exploration

After that, the project moves toward:

Coverage
↓
Bug Detection
↓
Bug Reproduction
↓
Test Generation
↓
Persistent Learning
↓
Local AI Planning

This roadmap is the master reference for the scope, scale, development order, and long-term purpose of the Free Form Testing Agent.